

Software Engineering and Project Management(BCS502)

Module – 1

1 Define Software and Explain the Nature of software?

Software is: (1) instructions (computer programs) that when executed provide desired features, function, and performance;

(2) data structures that enable the programs to adequately manipulate information, and

(3) descriptive information in both hard copy and virtual forms that describes the operation and use of the programs

Computer software continues to be the single most important technology on the world stage. Today, software takes on a dual role. It is a product, and at the same time, the vehicle for delivering a product.

- as a product, it delivers the computing potential embodied by computer hardware or more broadly, by a network of computers that are accessible by local hardware.
- as the vehicle used to deliver the product, software acts as the basis for the control of the computer (operating systems), the communication of information (networks), and the creation and control of other programs.
- Software delivers the most important product of our time—information. The role of computer software has undergone significant change over the last half-century.

2 Explain Different Software Application Domains.

Seven broad categories of computer software present continuing challenges for software engineers

- **System software** -- a collection of programs written to service other programs. Some system software (e.g., compilers, editors, and file management utilities) processes complex, but determinate, information structures. Other systems applications (e.g. Operating system components, drivers, networking software, telecommunications processors) process largely indeterminate data.

- **Application Software** -- stand-alone programs that solve a specific business need. Applications in this area process business or technical data in a way that facilitates business operations or management/technical decision making.

- **Engineering/scientific software** -- characterized by “number crunching” algorithms. Applications range from astronomy to volcanology, from automotive stress analysis to space shuttle orbital dynamics, and from molecular biology to automated manufacturing.

- **Embedded software** -- resides within a product or system and is used to implement and control features and functions for the end user and for the system itself.

- **Product-line software** -- designed to provide a specific capability for use by many different customers. Product-line software can focus on a limited and esoteric marketplace (e.g., inventory control products) or address mass consumer markets (e.g., word processing, spreadsheets, computer graphics etc)

- **Web Applications** -- his network-centric software category spans a wide array of applications. WebApps are evolving into sophisticated computing environments that not only provide stand-alone features, computing functions, and content to the end user, but also are integrated with corporate databases and business applications

- **Artificial Intelligence software** -- makes use of non-numerical algorithms to solve complex problems that are not amenable to computation or straightforward analysis. Applications within this area include robotics, expert systems, pattern recognition (image and voice), artificial neural networks, theorem proving, and game playing.

3 Explain THE UNIQUE NATURE OF WEBAPPS

In the early days of the World Wide Web, websites were just a set of linked hypertext files which presented information using text and limited graphics. The augmentation of HTML by development tools like Java, XML enabled web engineers to provide computing capability along with informational content.

Web-based systems and applications (WebApps) are sophisticated tools that not only present stand-alone information but also integrate databases and business applications. Web-based systems and applications “involve a mixture between print publishing and software development, between marketing and computing, between internal communications and external relations, and between art and technology.”

The following are the common attributes for WebApps:

- **Network intensiveness:** A WebApp resides on a network (Internet or Intranet) and must serve the needs of a diverse community of clients.
- **Concurrency:** A large number of users may access the WebApp at one time
- **Unpredictable load:** The number of users of the WebApp may vary by orders of magnitude from day to day.
- **Performance:** If a WebApp user must wait too long, he or she may decide to go elsewhere.
- **Availability:** Although the expectation of 100 percent availability is unreasonable, users of popular WebApps often demand access on a 24/7/365 basis.
- **Data driven:** The primary function of many WebApps is to use hypermedia to present text, graphics, audio, and video content to the end user
- **Content sensitive:** The quality and aesthetic nature of content remains an important determinant of the quality of a WebApp
- **Continuous evolution:** Unlike conventional application software that evolves over a series of planned, chronologically spaced releases, Web applications evolve continuously.

- **Immediacy:** WebApps often exhibit a time-to-market that can be a matter of a few days or weeks.
- **Security:** Because WebApps are available via network access sensitive content must be protected and secure modes of data transmission must be provided.
- **Aesthetics:** An undeniable part of the appeal of a WebApp is its look and feel.

4 Define Software Engineering and explain its features in detail.

Software Engineering Definition: The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software

Features Software Engineering:

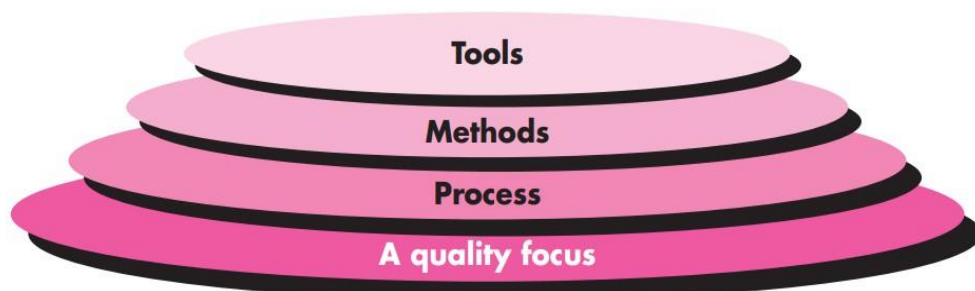
In order to build software that is ready to meet the challenges of the twenty-first century, you must recognize a few simple realities:

- It follows that a concerted effort should be made to understand the problem before a software solution is developed.
- It follows that design becomes a pivotal activity.
- It follows that software should exhibit high quality.
- It follows that software should be maintainable.

These simple realities lead to one conclusion: *software in all of its forms and across all of its application domains should be engineered.* And that leads us to the topic of this book—*software engineering.*

FIGURE 1.3

Software
engineering
layers



- Process defines a framework that must be established for effective delivery of software engineering technology.
- The **software process** forms the basis for management control of software projects and establishes the context in which technical methods are applied, work products (models, documents, data, reports, forms, etc.) are produced, milestones are established, quality is ensured, and change is properly managed.
- Software engineering methods provide the technical how-to's for building software.

- **Methods** encompass a broad array of tasks that include communication, requirements analysis, design modeling, program construction, testing, and support. Software engineering methods rely on a set of basic principles that govern each area of the technology and include modeling activities and other descriptive techniques.
- Software engineering **tools** provide automated or semi-automated support for the process and the methods.
- When tools are integrated so that information created by one tool can be used by another, a system for the support of software development, called computer-aided software engineering, is established.

5 Explain THE SOFTWARE Engineering Process in detail.

A process is a collection of activities, actions, and tasks that are performed when some work product is to be created.

- An **activity** strives to achieve a broad objective (e.g. communication with stakeholders) and is applied regardless of the application domain, size of the project, complexity of the effort, or degree of rigor with which software engineering is to be applied.
- An **action** (e.g., architectural design) encompasses a set of tasks that produce a major work product (e.g., an architectural design model).
- A **task** focuses on a small, but well-defined objective (e.g., conducting a unit test) that produces a tangible outcome.

A process is not a rigid prescription for how to build computer software. Rather, it is an adaptable approach that enables the people doing the work (the software team) to pick and choose the appropriate set of work actions and tasks. The intent is always to deliver software in a timely manner and with sufficient quality to satisfy those who have sponsored its creation and those who will use it.

A *process framework* establishes the foundation for a complete software engineering process by identifying a small number of *framework activities* that are applicable to all software projects, regardless of their size or complexity.

Communication. Before any technical work can commence, it is critically important to communicate and collaborate with the customer, to understand objectives of the project and to gather requirements.

Planning. A software project is a complicated journey, and the planning activity creates a “map” that helps guide the team as it makes the journey. The map—called a software project plan—defines the software engineering work by describing the technical tasks to be conducted, the risks that are likely, the resources that will be required, the work products to be produced, and a work schedule.

Modeling. Creating models to better understand software requirements and the design that will achieve those requirements.

Construction. : This activity combines code generation and the testing that is required to uncover errors in the code.

Deployment. The software is delivered to the customer who evaluates the delivered product and provides feedback based on the evaluation.

These five generic framework activities can be used during the development of small, simple programs, the creation of large Web applications, and for the engineering of large, complex computer-based systems.

- Typical **umbrella activities** include:
 - **Software project tracking and control-** allows the software team to assess progress against the project plan and take any necessary action to maintain the schedule.
 - **Risk management-**assesses risks that may affect the outcome of the project or the quality of the product.
 - **Software quality assurance-** defines and conducts the activities required to ensure software quality.
 - **Technical reviews-** assesses software engineering work products in an effort to uncover and remove errors before they are propagated to the next activity.
 - **Measurement-**defines and collects process, project, and product measures that assist the team in delivering software that meets stakeholders' needs.
 - **Software configuration management-** manages the effects of change throughout the software process.
 - **Reusability management-** defines criteria for work product reuse (including software components) and establishes mechanisms to achieve reusable components.
 - **Work product preparation and production-** encompasses the activities required to create work products such as models, documents, logs, forms, and lists.

6 Explain the Essence of Software Engineering Practice

Understand the problem (communication and analysis).

- Who has a stake in the solution to the problem? That is, who are the stakeholders?
- What are the unknowns? What data, functions, and features are required to properly solve the problem?
- Can the problem be compartmentalized? Is it possible to represent smaller problems that may be easier to understand?
- Can the problem be represented graphically? Can an analysis model be created?

Plan a solution (modeling and software design).

- *Have you seen similar problems before?*
- *Has a similar problem been solved? If so, are elements of the solution reusable?*
- *Can sub problems be defined? If so, are solutions readily apparent for the sub problems?*
- *Can you represent a solution in a manner that leads to effective implementation? Can a design model be created?*

Carry out the plan (code generation).

- *Does the solution conform to the plan? Is source code traceable to the design model?*
- *Is each component part of the solution provably correct? Have the design and code been reviewed*

Examine the result.

- *Is it possible to test each component part of the solution? Has a reasonable testing strategy been implemented?*
- *Does the solution produce results that conform to the data, functions, and features that are required? Has the software been validated against all stakeholder requirements?*

7 Explain David Hooker seven principles that focus on software engineering practice as a whole

1. The First Principle: The Reason It All Exists A software system exists for one reason: to provide value to its users. All decisions should be made with this in mind.

2. The Second Principle: KISS (Keep It Simple, Stupid!) All design should be as simple as possible, but no simpler. This facilitates having a more easily understood and easily maintained system.

3. The Third Principle: Maintain the Vision A clear vision is essential to the success of a software project. Without one, a project almost unfailingly ends up being “of two [or more] minds” about itself.

4. The Fourth Principle: What You Produce, Others Will Consume always specify, design, and implement knowing someone else will have to understand what you are doing. The audience for any product of software development is potentially large.

5. The Fifth Principle: Be Open to the Future Never design yourself into a corner. Always ask “what if,” and prepare for all possible answers by creating systems that solve the general problem, not just the specific one. 6. The Sixth Principle: Plan Ahead for Reuse Planning ahead for reuse reduces the cost and increases the value of both the reusable components and the systems into which they are incorporated.

7. The Seventh principle: Think! Placing clear, complete thought before action almost always produces better results. When you think about something, you are more likely to do it right. You also gain knowledge about how to do it right again.

8 Explain Different Software MYTHS.

• Software myths: erroneous beliefs about software and the process that is used to build it
We categorize myths from three different perspectives.

Management myths

- **Myth:** *We already have a book that's full of standards and procedures for building software. Won't that provide my people with everything they need to know?*
- **Reality:** **The book of standards may very well exist, but is it used?** Are software practitioners aware of its existence? Does it reflect modern software engineering practice? Is it complete? Is it adaptable? Is it streamlined to improve time-to-delivery while still maintaining a focus on quality? In many cases, the answer to all of these questions is “no.”
- **Myth:** *If we get behind schedule, we can add more programmers and catch up (sometimes called the “Mongolian horde” concept).*
- **Reality:** Software development is not a mechanistic process like manufacturing. In the words of Brooks [Bro95]: “adding people to a late software project makes it later.” At first, this statement may seem counterintuitive. However, as new people are added, people who were working must spend time educating the newcomers, thereby reducing the amount of time spent on productive development effort. People can be added but only in a planned and well-coordinated manner.
- **Myth:** *If I decide to outsource the software project to a third party, I can just relax and let that firm build it.*
- **Reality:** If an organization does not understand how to manage and control software projects internally, it will invariably struggle when it outsources software projects.

Customer myths.

- **Myth:** *A general statement of objectives is sufficient to begin writing programs—we can fill in the details later*
- **Reality:** Although a comprehensive and stable statement of requirements is not always possible, an ambiguous “statement of objectives” is a recipe for disaster. Unambiguous requirements are developed only through effective and continuous communication between customer and developer
- **Myth:** Software requirements continually change, but change can be easily accommodated because software is flexible.
- **Reality:** It is true that software requirements change, but the impact of change varies with the time at which it is introduced. When requirements changes are requested early the cost impact is relatively small.¹⁶ However, as time passes, the cost impact grows rapidly

Practitioner's myths

- **Myth:** Once we write the program and get it to work, our job is done.
- **Reality:** Someone once said that “the sooner you begin ‘writing code,’ the longer it’ll take you to get done.” Industry data indicate that between 60 and 80 percent of all effort expended on software will be expended after it is delivered to the customer for the first time.
- **Myth:** The only deliverable work product for a successful project is the working program.
- **Reality:** A working program is only one part of a software configuration that includes many elements. A variety of work products (e.g., models, documents, plans) provide a foundation for successful engineering and, more important, guidance for software support.



www.sjcit.ac.in

|| Jai Sri Gurudev ||
Sri Adichunchanagiri Shikshana Trust®

SJC INSTITUTE OF TECHNOLOGY

An Autonomous Institution under VTU from 2024-25

AICTE Approved, Accredited by NBA [CSE, ISE, ECE, ME, CV, AE] and NAAC with A+ Grade, QS I-Gauge Gold rated
P.B. No. 20, B.B. Road, Chikkaballapur - 562 101, Karnataka.



Estd. 1986

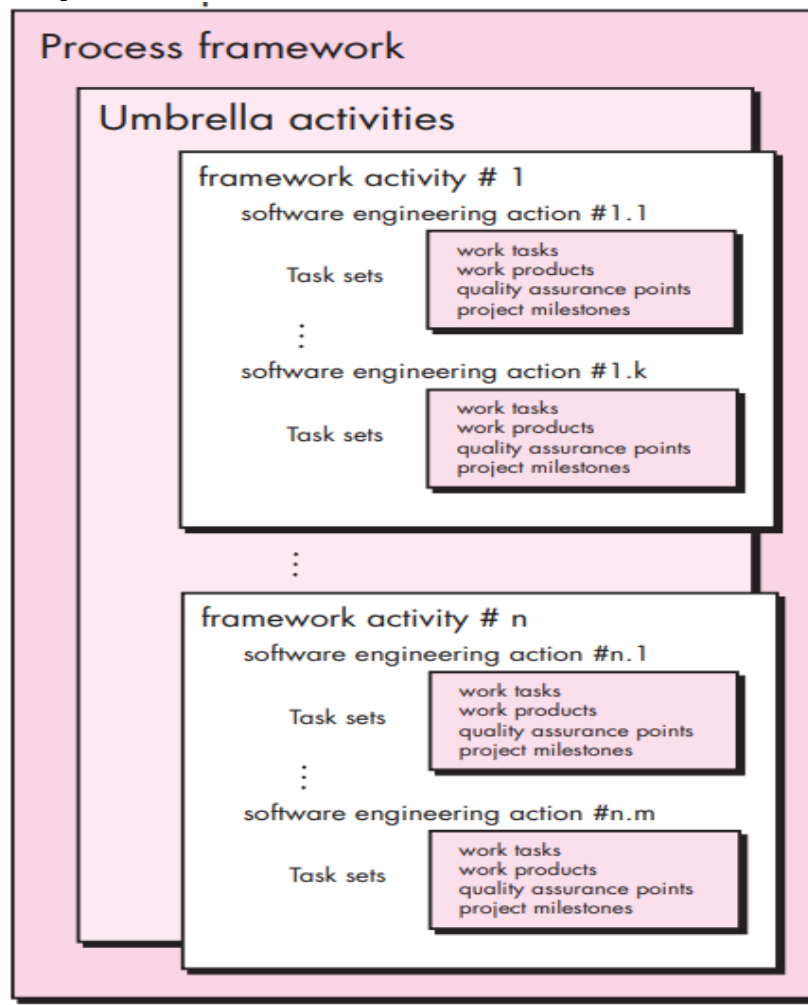
Myth: Software engineering will make us create voluminous and unnecessary documentation and will invariably slow us down.

- **Reality:** Software engineering is not about creating documents. It is about creating a quality product. Better quality leads to reduced rework. And reduced rework results in faster delivery times.

9 Explain Generic Software Process Model in details and different types.

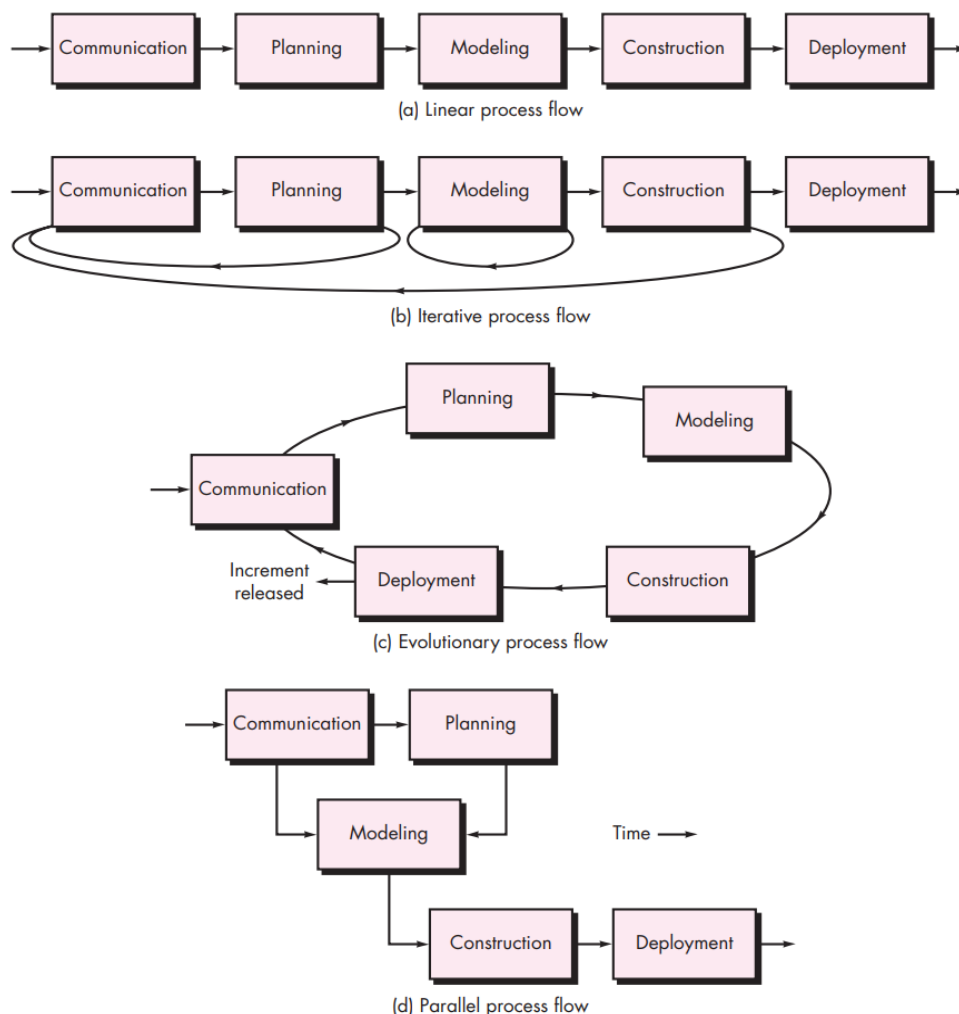
A process was defined as a collection of work activities, actions, and tasks that are performed when some work product is to be created.

- Each of these activities, actions, and tasks reside within a framework or model that defines their relationship with the process and with one another.
- A generic process framework for software engineering defines five framework activities — **communication, planning, modeling, construction, and deployment.**
- Each framework activity is populated by a set of software engineering actions.
- Each software engineering action is defined by a task set that identifies the work tasks that are to be completed, the work products that will be produced, the quality assurance points that will be required, and the milestones that will be used to indicate progress.
- One important aspect of the software process called process flow—describes how the framework activities and the actions and tasks that occur within each framework activity are organized with respect to sequence and time



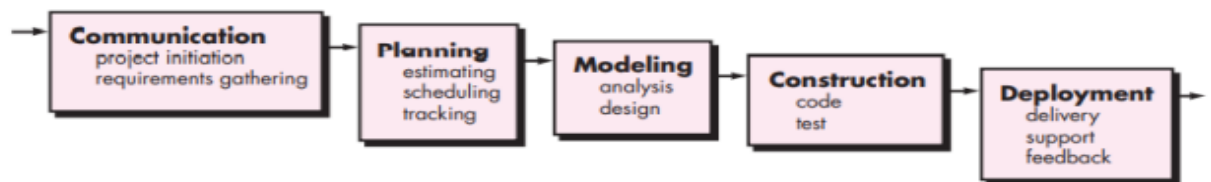
- One important aspect of the software process is **process flow**—describes how the framework activities and the actions and tasks that occur within each framework activity are organized with respect to sequence and time and is illustrated in Figure 2.2.
- A **linear process flow** executes each of the five framework activities in sequence, beginning with communication and culminating with deployment (Figure 2.2a).
- An **iterative process flow** repeats one or more of the activities before proceeding to the next (Figure 2.2b).
- An **evolutionary process flow** executes the activities in a “circular” manner. Each circuit through the five activities leads to a more complete version of the software (Figure 2.2c).
- A **parallel process flow** (Figure 2.2d) executes one or more activities in parallel with other activities (e.g., modeling for one aspect of the software might be executed in parallel with construction of another aspect of the software).

FIGURE 2.2 Process flow



10 Explain in details Waterfall model.

- There are times when the requirements for a problem are well understood—when work flows from communication through deployment in a reasonably linear fashion.
- The waterfall model, sometimes called the classic life cycle, suggests a systematic, sequential approach to software development that begins with customer specification of requirements and progresses through planning, modeling, construction, and deployment, culminating in ongoing support of the completed software.
- The original waterfall model proposed by Winston Royce made provision for “feedback loops,” the vast majority of organizations that apply this process model treat it as if it were strictly linear.

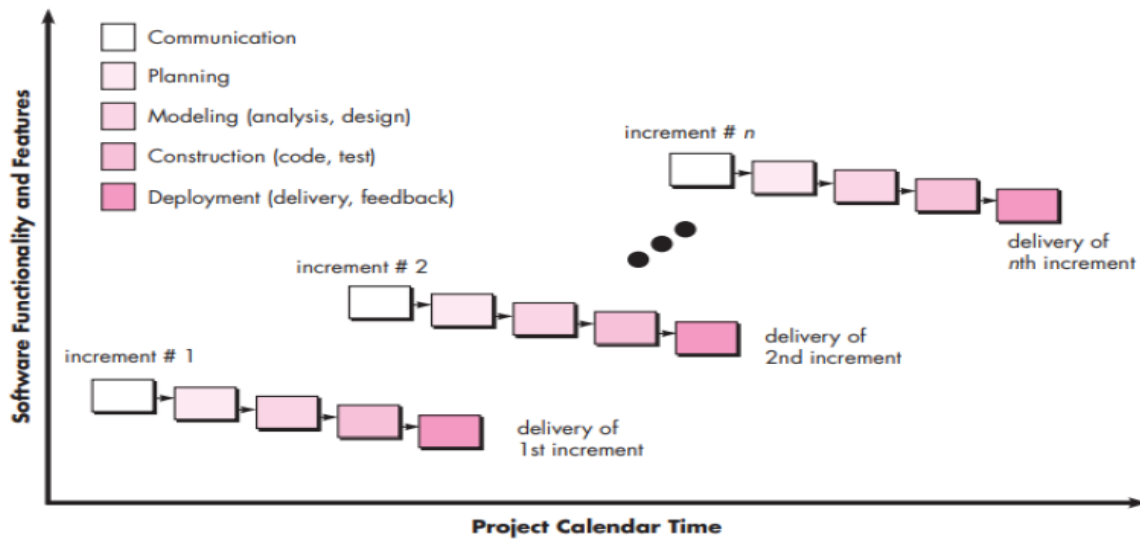


Waterfall Model or Classical lifecycle model

- The waterfall model is the oldest paradigm for software engineering. the problems that are sometimes encountered when the waterfall model is applied are:
 - 1 Real projects rarely follow the sequential flow that the model proposes. Although the linear model can accommodate iteration, it does so indirectly. As a result, changes can cause confusion as the project team proceeds.
 2. It is often difficult for the customer to state all requirements explicitly. The waterfall model requires this and has difficulty accommodating the natural uncertainty that exists at the beginning of many projects.
 3. The customer must have patience. A working version of the program(s) will not be available until late in the project time span. A major blunder, if undetected until the working program is reviewed, can be disastrous.
 4. The linear nature of the classic life cycle leads to “blocking states” in which some project team members must wait for other members of the team to complete dependent tasks. In fact, the time spent waiting can exceed the time spent on productive work.
- The waterfall model is often inappropriate today, as software work is fast-paced and subject to a never-ending stream of changes.

11 Explain in detail Incremental Process Models

- The incremental model combines elements of linear and parallel process flows.
- The incremental model applies linear sequences in a staggered fashion as calendar time progresses.
- Each linear sequence produces deliverable “increments” of the software in a manner that is similar to the increments produced by an evolutionary process flow.



12 Explain in detail prototyping model

- Prototype is defined as first or preliminary form using which other forms are copied or derived.
- Prototype model is a set of general objectives for software.
- It does not identify the requirements like detailed input, output.
- It is software working model of limited functionality.
- In this model, working programs are quickly produced.

Advantages of Prototyping Model

- Prototype model need not know the detailed input, output, processes, adaptability of operating system and full machine interaction.
- In the development process of this model users are actively involved.
- The development process is the best platform to understand the system by the user.
- Errors are detected much earlier.
- Gives quick user feedback for better solutions.
- It identifies the missing functionality easily. It also identifies the confusing or difficult functions.

Disadvantages of Prototyping Model:

- The client involvement is more and it is not always considered by the developer.
- It is a slow process because it takes more time for development.
- Many changes can disturb the rhythm of the development team.
- It is a thrown away prototype when the users are confused with it.

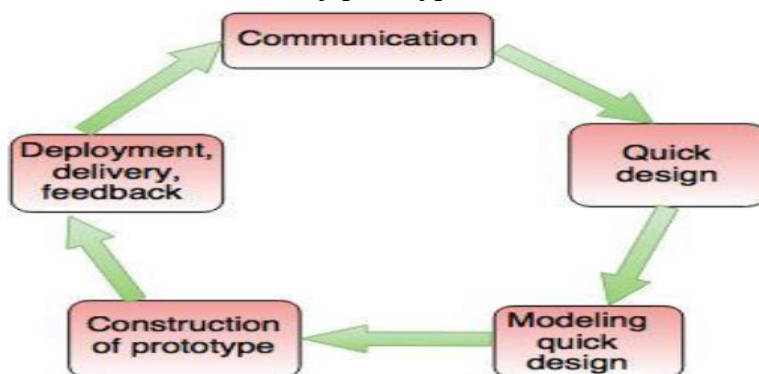
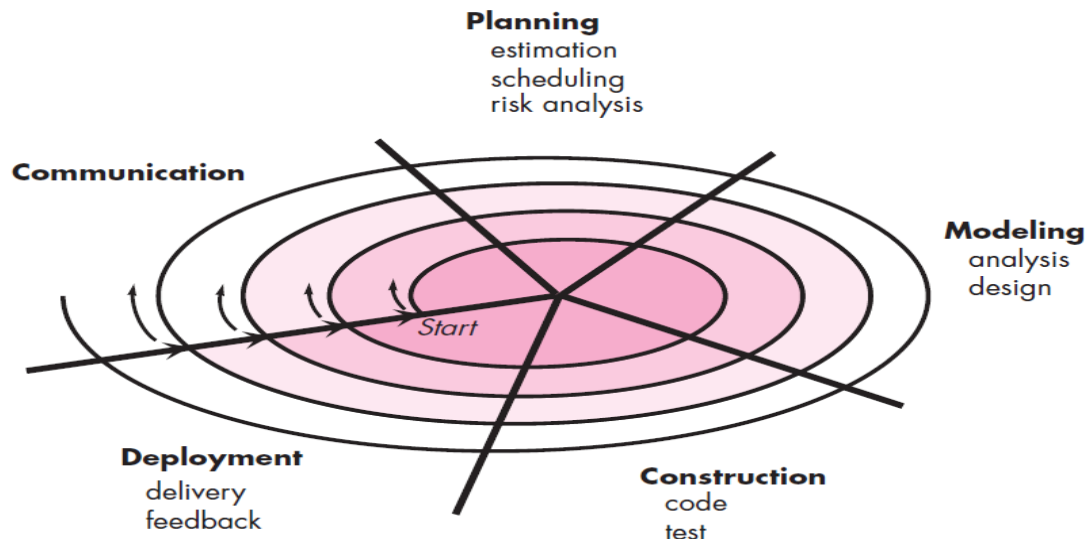


Fig. - The Prototyping Model



13 Explain in detail Spiral model

- Spiral model is a risk driven process model.
- It is used for generating the software projects.
- In spiral model, an alternate solution is provided if the risk is found in the risk analysis, then alternate solutions are suggested and implemented.
- It is a combination of prototype and sequential model or waterfall model.
- In one iteration all activities are done, for large project's the output is small.



Advantages of Spiral Model

- It reduces high amount of risk.
- It is good for large and critical projects.
- It gives strong approval and documentation control.
- In spiral model, the software is produced early in the life cycle process.

Disadvantages of Spiral Model

- It can be costly to develop a software model.
- It is not used for small projects.

14 Explain Concurrent Development Model

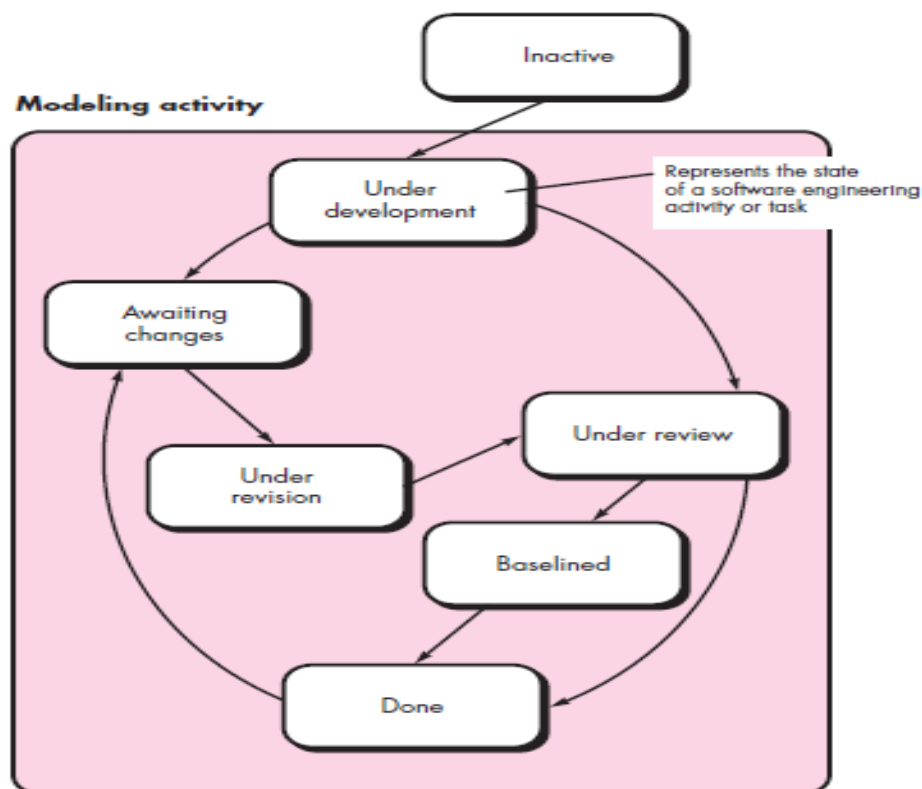
- The concurrent development model is called as concurrent model.
- The communication activity has completed in the first iteration and exits in the awaiting changes state.
- The modeling activity completed its initial communication and then go to the underdevelopment state.
- If the customer specifies the change in the requirement, then the modeling activity moves from the under development state into the awaiting change state.
- The concurrent process model activities moving from one state to another state

Advantages of the concurrent development model

- This model is applicable to all types of software development processes.
- It is easy for understanding and use.
- It gives immediate feedback from testing.
- It provides an accurate picture of the current state of a project.

Disadvantages of the concurrent development model

- It needs better communication between the team members. This may not be achieved all the time.
- It requires to remember the status of the different activities.



15 Explain Legacy software

- Legacy software is an older version of a computer program or application that is still in use, even after newer versions are available.
- Legacy systems are often outdated, but still meet the needs they were originally designed for.
- Legacy systems are often critical to day-to-day operations, but can be challenging to maintain and support.
- They may not be compatible with newer operating systems or hardware, and they don't receive technical support or updates from their developers.
- This can make them vulnerable to data breaches, which can cost organizations millions of dollars.
- A common legacy system example is Microsoft's Windows 7, which was released in 2009 and was no longer supported after 2020.

However, as time passes, legacy systems often evolve for one or more of the following reasons:

- The software must be adapted to meet the needs of new computing environments or technology.
- The software must be enhanced to implement new business requirements.
- The software must be extended to make it interoperable with other more modern systems or databases.
- The software must be re-architected to make it viable within a network environment.

16 Explain IDENTIFYING A TASK SET

- A **task set** defines the actual work to be done to accomplish the objectives of a software engineering action.
- For example, elicitation (more commonly called “requirements gathering”) is an important software engineering action that occurs during the communication activity.
- The goal of requirements gathering is to understand what various stakeholders want from the software that is to be built.
- For a small, relatively simple project, the task set for requirements gathering might look like this:
 1. Make a list of stakeholders for the project.
 2. Invite all stakeholders to an informal meeting.
 3. Ask each stakeholder to make a list of features and functions required.
 4. Discuss requirements and build a final list.
 5. Prioritize requirements.
 6. Note areas of uncertainty.

17 Explain Process Patterns with in detail.

- ✓ A process pattern describes a process-related problem that is encountered during software

Engineering work, identifies the environment in which the problem has been encountered, and suggests one or more proven solutions to the problem.

- ✓ A process pattern provides you with a template - a consistent method for describing problem solutions within the context of the software process.
- ✓ By combining patterns, a software team can solve problems and construct a process that best meets the needs of a project.
- ✓ Ambler [Amb98] has proposed a template for describing a process pattern:

Pattern Name: The pattern is given a meaningful name describing it within the context of the software process (e.g., Technical Reviews).

Forces: The environment in which the pattern is encountered and the issues that make the problem visible and may affect its solution.

Type: The pattern type is specified. There are three types:

1. **Stage pattern**—defines a problem associated with a framework activity for the process. Since a framework activity encompasses multiple actions and work tasks, a stage pattern incorporates multiple task patterns (see the following) that are relevant to the stage (framework activity). An example of a stage pattern might be Establishing

Communication. This pattern would incorporate the task pattern Requirements Gathering and others.

2. **Task pattern**—defines a problem associated with a software engineering action or work task and relevant to successful software engineering practice (e.g., Requirements Gathering is a task pattern).
3. **Phase pattern**—define the sequence of framework activities that occurs within the process, even when the overall flow of activities is iterative in nature. An example of a phase pattern might be Spiral Model or Prototyping.

Initial context: Describes the conditions under which the pattern applies. Prior to the initiation of the pattern:

- (1) What organizational or team-related activities have already occurred?
- (2) What is the entry state for the process?
- (3) What software engineering information or project information already exists?

For example, the Planning pattern (a stage pattern) requires that

- (1) customers and software engineers have established a collaborative communication;
- (2) successful completion of a number of task patterns [specified] for the Communication pattern has occurred; and
- (3) the project scope, basic business requirements, and project constraints are known.

Problem. The specific problem to be solved by the pattern.

Solution. Describes how to implement the pattern successfully. This section describes how the initial state of the process (that exists before the pattern is implemented) is modified as a consequence of the initiation of the pattern. It also describes how software engineering information or project information that is available before the initiation of the pattern is transformed as a consequence of the successful execution of the pattern.

Resulting Context: Describes the conditions that will result once the pattern has been successfully implemented.

Upon completion of the pattern:

- (1) What organizational or team-related activities must have occurred?
- (2) What is the exit state for the process?
- (3) What software engineering information or project information has been developed?

Related Patterns: Provide a list of all process patterns that are directly related to this one. This may be represented as a hierarchy or in some other diagrammatic form.

For example, the stage pattern Communication encompasses the task patterns: Project Team, Collaborative Guidelines, Scope Isolation, Requirements Gathering, Constraint Description, and Scenario Creation.

Known Uses and Examples: Indicate the specific instances in which the pattern is applicable. For example, Communication is mandatory at the beginning of every software project, is recommended throughout the software project, and is mandatory once the deployment activity is under way.

An Example Process Pattern

Pattern name: Requirements Unclear

Intent: This pattern describes an approach for building a model (a prototype) that can be assessed iteratively by stakeholders in an effort to identify or solidify software requirements.

Type: Phase pattern.

Initial context: The following conditions must be met prior to the initiation of this pattern:

- (1) stakeholders have been identified;

- (2) a mode of communication between stakeholders and the software team has been established;
- (3) the overriding software problem to be solved has been identified by stakeholders;
- (4) an initial understanding of project scope, basic business requirements, and project constraints has been developed.

Problem. Requirements are hazy or non-existent, yet there is clear recognition that there is a problem to be solved, and the problem must be addressed with a software solution. Stakeholders are unsure of what they want; that is, they cannot describe software requirements in any detail.

Resulting context: A software prototype that identifies basic requirements (e.g., modes of interaction, computational features, processing functions) is approved by stakeholders.

Following this, (1) the prototype may evolve through a series of increments to become the production software or (2) the prototype may be discarded and the production software built using some other process pattern. Related patterns. The following patterns are related to this pattern: Customer Communication, Iterative Design, Iterative Development, Customer Assessment, and Requirement Extraction.

Known uses and examples: Prototyping is recommended when requirements are uncertain.

18. Explain PROCESS ASSESSMENT AND IMPROVEMENT

A number of different approaches to software process assessment and improvement have been proposed over the past few decades:

- **Standard CMMI Assessment Method for Process Improvement (SCAMPI)**-provides a five- step process assessment model that incorporates five phases: initiating, diagnosing, establishing, acting, and learning. The SCAMPI method uses the SEI CMMI as the basis for assessment.
- **CMM-Based Appraisal for Internal Process Improvement (CBA IPI)** - provides a diagnostic technique for assessing the relative maturity of a software organization; uses the SEI CMM as the basis for the assessment [Dun01].
- **SPICE (ISO/IEC15504)**—a standard that defines a set of requirements for software process assessment. The intent of the standard is to assist organizations in developing an objective evaluation of the efficacy of any defined software process [ISO08].
- **ISO 9001:2000 for Software**—a generic standard that applies to any organization that wants to improve the overall quality of the products, systems, or services that it provides. Therefore, the standard is directly applicable to software organizations and companies [Ant06].

19. Explain V Model in the context of Waterfall Model.

- A variation in the representation of the waterfall model is called the V-model.
- Represented in Figure 2.4, the V-model [Buc99] depicts the relationship of quality assurance actions to the actions associated with communication, modeling, and early construction activities.
- As a software team moves down the left side of the V, basic problem requirements are refined into progressively more detailed and technical representations of the problem and its solution.
- Once code has been generated, the team moves up the right side of the V, essentially performing a series of tests (quality assurance actions) that validate each of the models created as the team

- In reality, there is no fundamental difference between the classic life cycle and the V-model.
- The V-model provides a way of visualizing how verification and validation actions are applied to earlier engineering work.
- The waterfall model is the oldest paradigm for software engineering.
- However, over the past three decades, criticism of this process model has caused even ardent supporters to question its efficacy.
- Problems that are encountered when the waterfall model is applied are:
 1. Real projects rarely follow the sequential flow that the model proposes. Although the linear model can accommodate iteration, it does so indirectly. As a result, changes can cause confusion as the project team proceeds.
 2. It is often difficult for the customer to state all requirements explicitly. The waterfall model requires this and has difficulty accommodating the natural uncertainty that exists at the beginning of many projects.
 3. The customer must have patience. A working version of the program(s) will not be available until late in the project time span. A major blunder, if undetected until the working program is reviewed, can be disastrous.

```
graph TD; RM[Requirements modeling] --> AD[Architectural design]; AD --> CD[Component design]; CD --> CG[Code generation]; CG --> ES[Executable software]; UT[Unit testing] --> IT[Integration testing]; IT --> ST[System testing]; ST --> AT[Acceptance testing]; AT --> ES; AT --> RM; ST --> AD; IT --> CD; UT --> CG;
```

20. Explain SPECIALIZED PROCESS MODELS

Specialized process models take on many of the characteristics of one or more of the traditional models.

Component-Based Development

- Commercial off-the-shelf (COTS) software components, developed by vendors who offer them as products, provide targeted functionality with well-defined interfaces that enable the component to be integrated into the software that is to be built.
- The component-based development model incorporates many of the characteristics of the spiral model. It is evolutionary in nature [Nie92], demanding an iterative approach to the creation of software. However, the component-based development model constructs applications from prepackaged software components.
- Modeling and construction activities begin with the identification of candidate components. These components can be designed as either conventional software modules or object-oriented classes or packages of classes.
- Regardless of the technology that is used to create the components, the component-based development model incorporates the following steps
 1. Available component-based products are researched and evaluated for the application domain in question.
 2. Component integration issues are considered.
 3. A software architecture is designed to accommodate the components.
 4. Components are integrated into the architecture.
 5. Comprehensive testing is conducted to ensure proper functionality.
- The component-based development model leads to software reuse, and reusability provides software engineers with a number of measurable benefits.

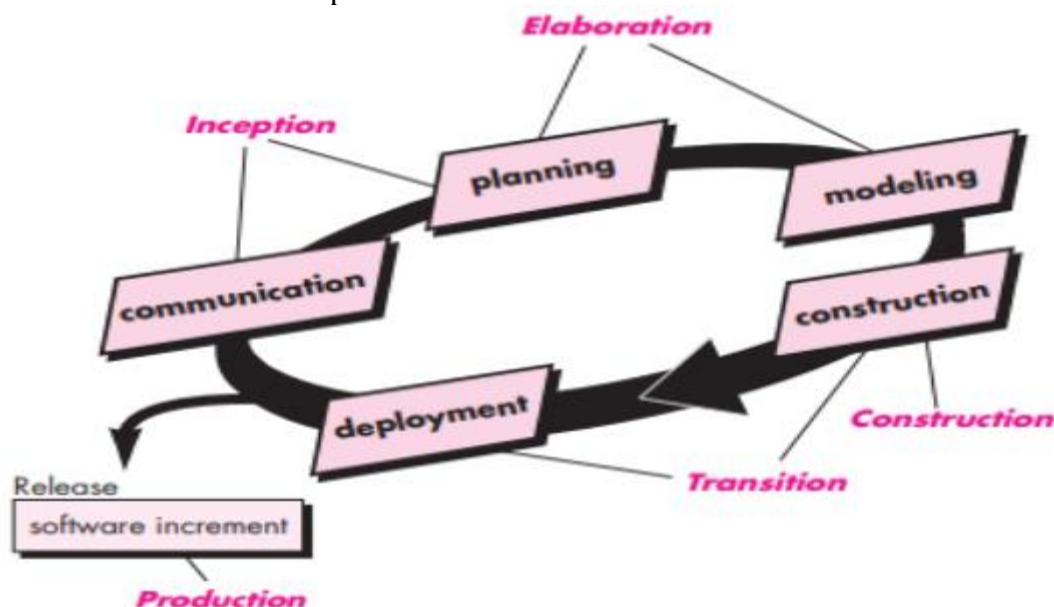
The Formal Methods Model

- The formal methods model encompasses a set of activities that leads to formal mathematical specification of computer software.
- Formal methods enable you to specify, develop, and verify a computer-based system by applying a rigorous, mathematical notation.
- A variation on this approach, called cleanroom software engineering, is currently applied by some software development organizations.
- When formal methods are used during development, they provide a mechanism for eliminating many of the problems that are difficult to overcome using other software engineering paradigms.
- Ambiguity, incompleteness, and inconsistency can be discovered and corrected more easily—not through ad hoc review, but through the application of mathematical analysis.
- When formal methods are used during design, they serve as a basis for program verification and therefore enable you to discover and correct errors that might otherwise go undetected.
- Although not a mainstream approach, the formal methods model offers the promise of defect-free software.
- Yet, concern about its applicability in a business environment has been voiced:

- The development of formal models is currently quite time consuming and expensive
- Because few software developers have the necessary background to apply formal methods, extensive training is required.
- It is difficult to use the models as a communication mechanism for technically unsophisticated customers.
- Aspect-Oriented Software Development
- Aspectual requirements define those crosscutting concerns that have an impact across the software architecture.
- Aspect-oriented software development (AOSD), often referred to as aspect- oriented programming (AOP), is a relatively new software engineering paradigm that provides a process and methodological approach for defining, specifying, designing, and constructing aspects—“mechanisms beyond subroutines and inheritance for localizing the expression of a crosscutting concern”.

21 Explain Unified process Model

The Unified Process is an attempt to draw on the best features and characteristics of traditional software process models, but characterize them in a way that implements many of the best principles of agile software development. The Unified Process recognizes the importance of customer communication and streamlined methods for describing the customer’s view of a system. It emphasizes the important role of software architecture and “helps the architect focus on the right goals, such as understand ability, reliance to future changes, and reuse. It suggests a process flow that is iterative and incremental, providing the evolutionary feel that is essential in modern software development.



Inception: The inception phase of the UP encompasses both customer communication and planning activities. By collaborating with stakeholders, business requirements for the software are identified; a rough architecture for the system is proposed; and a plan for the iterative, incremental nature of the ensuing project is developed.

Planning : Planning identifies resources, assesses major risks, defines a schedule, and establishes a basis for the phases that are to be applied as the software increment is developed.

Elaboration: The elaboration phase encompasses the communication and modelling activities of the generic process model. Elaboration refines and expands the preliminary use cases that were developed as part of the inception phase and expands the architectural representation to include five different views of the software—the use case model, the requirements model, the design model, the implementation model, and the deployment model. Elaboration refines and expands the preliminary use cases that were developed as part of the inception phase and expands the architectural representation to include five different views of the software—the use case model, the requirements model, the design model, the implementation model, and the deployment model.

Construction: The construction phase of the UP is identical to the construction activity defined for the generic software process. Using the architectural model as input, the construction phase develops or acquires the software components that will make each use case operational for end users. To accomplish this, requirements and design models that were started during the elaboration phase are completed to reflect the final version of the software increment. All necessary and required features and functions for the software increment are then implemented in source code. As components are being implemented, unit tests are designed and executed for each. In addition, integration activities are conducted. Use cases are used to derive a suite of acceptance tests that are executed prior to the initiation of the next unified process phase.

Transition: The transition phase of the UP encompasses the latter stages of the generic construction activity and the first part of the generic deployment activity. Software is given to end users for beta testing and user feedback reports both defects and necessary changes. In addition, the software team creates the necessary support information that is required for the release. At the conclusion of the transition phase, the software increment becomes a usable software release.

Production: The production phase of the UP coincides with the deployment activity of the generic process. During this phase, the ongoing use of the software is monitored, support for the operating environment (infrastructure) is provided, and defect reports and requests for changes are submitted and evaluated.

22 Explain PSP and TSP

PERSONAL PROCESS MODELS

- In software engineering, personal software process (PSP) models focus on improving the performance of individuals, while team software process models focus on improving the performance of a group.
- PSP is a bottom-up approach that includes tools and techniques to help software engineers improve the quality of their work.
- PSP provides a framework that helps engineers analyze their performance and achieve goals.

- **PSP** process model defines five framework activities: planning, high-level design, high-level design review, development, and postmortem. It stresses the need to identify errors early and to understand the types of errors.
- **Planning:** it isolates requests. And a project schedule is created.
- **High-level design:** Prototypes are built when uncertainty exists.
- **High-level design review:** Formal verification methods are applied to uncover errors in the design.
- **Development:** Code is generated, reviewed, compiled, and tested.
- **Postmortem:** using the measures and metrics collected, the effectiveness of the process is determined.

➤ **Team software process models** depend on a group of people.

The goal of TSP is to build a “self-directed” project team that organizes itself to produce high-quality s/w.

- Each project is “launched” using a “script” that defines the tasks to be accomplished.
- Teams are self-directed.
- Measurement is encouraged.
- Measures are analysed with the intent of improving the team process.